SQL Server JOINs

24 interview questions, Easy to Hard. Every query is answered, run against one small dataset, and explained step by step: how SQL Server actually executes it, where candidates go wrong, and what the interviewer asks next.

36 pages · Free ■□□ Easy · 8 ■■□ Intermediate · 8 ■■■ Hard · 8

Data with Deepankar

Instagram @datawithdeepankar · YouTube Deepankar Pathak

Can you answer these?

Five questions that look easy and catch most candidates. Try them before you turn the page. Every answer is inside.

---

1 T1 holds 1, 1, 2, NULL. T2 holds 1, 1, 1, 3, NULL. How many rows does T1 INNER JOIN T2 ON T1.val = T2.val return?

ANSWER · Q09

2 WHERE dept_id NOT IN (SELECT dept_id FROM Employees) returns nothing, even though Finance has no employees. Why?

ANSWER · Q22

3 Your LEFT JOIN suddenly returns fewer rows than the left table. What did you put in WHERE?

ANSWER · Q10

4 An order worth 500 shows SUM = 1500 after a join. Where did the extra 1000 come from?

ANSWER · Q21

5 COUNT(*) says Finance has 1 employee. It has none. What went wrong?

ANSWER · Q08

How to use this guide

Don't just read the answers. For each question, predict the output and the row count first, then check. Interviewers rarely ask "what is a LEFT JOIN"; they ask "how many rows will this return, and why".

1. Run the setup script (next pages) in SQL Server Management Studio or Azure Data Studio.
2. Read the question, write your own query, and guess the row count.
3. Compare with the query, the output and the "How it executes" walkthrough.
4. Read the Trap and the Interviewer follow-up. That is where most candidates lose marks.

Every page has the same parts

Query	Written in T-SQL (SQL Server). Runs as-is on SQL Server 2016+; Q14's first form needs SQL Server 2022.
Output	The real result on the practice dataset, with the row count.
How it executes	The logical steps SQL Server follows, with row counts at each step.
Trap	The mistake interviewers are waiting for.
Follow-up	The next question you will probably get.

Contents

EASY

Q01	INNER JOIN — employees with their department
Q02	LEFT JOIN — keep every employee
Q03	Find departments with no employees
Q04	RIGHT JOIN — and why most people rewrite it
Q05	FULL OUTER JOIN — nothing gets lost
Q06	CROSS JOIN — every combination
Q07	SELF JOIN — employee and manager name
Q08	Count employees per department, including zero

INTERMEDIATE

Q09	The duplicate + NULL row-count puzzle
Q10	ON vs WHERE in a LEFT JOIN
Q11	Employees who earn more than their manager
Q12	Highest-paid employee in each department
Q13	Chaining joins: one INNER JOIN breaks your LEFT JOINs
Q14	Joining when the key can be NULL
Q15	Non-equi join — salary grades
Q16	Pairs of employees in the same department

HARD

Q17	Employees above their department average
Q18	Second-highest salary per department (no window functions)
Q19	CROSS APPLY vs OUTER APPLY — top earner per department
Q20	Recursive self join — full reporting chain
Q21	The fan-out trap — inflated SUMs
Q22	NOT IN vs NOT EXISTS vs LEFT JOIN — the NULL trap
Q23	Relational division — employees on ALL projects
Q24	Logical vs physical joins — what SQL Server really runs

The practice dataset

One small company. It is tiny on purpose, so you can verify every answer by hand. Notice the edge cases: **Kabir** has no department, **Finance** has no employees, **Aarav** has no manager, and **Priya** and **Ishaan** earn the same salary. Most join questions are really about these edge cases.

Employees

emp_id	emp_name	dept_id	manager_id	salary
101	Aarav	10	*NULL*	150,000
102	Priya	10	101	95,000
103	Rohan	20	101	80,000
104	Sneha	20	103	85,000
105	Kabir	*NULL*	101	55,000
106	Meera	30	101	70,000
107	Ishaan	10	102	95,000

Departments

dept_id	dept_name	city
10	Engineering	Jaipur
20	Sales	Delhi
30	HR	Pune
40	Finance	Mumbai

SalaryGrades

grade	min_sal	max_sal
A	100,000	200,000
B	75,000	99,999
C	50,000	74,999

Projects

project_id	project_name
1	Data Lake
2	CRM Revamp
3	Payroll Bot

EmployeeProjects

emp_id	project_id
101	1, 2, 3
102	1
103	2
104	2, 3
107	1, 2, 3

OrderItems

order_id	item	amount
5001	Laptop bag	300
5001	Mouse	200
5002	Monitor	400

Payments

payment_id	order_id	paid
1	5001	100
2	5001	150
3	5001	250
4	5002	400

T1 / T2

T1.val	T2.val
1	1
1	1
2	1
NULL	3
	NULL

EmployeeProjects is shown grouped for space; the table stores one row per employee per project (10 rows).

Setup script

Create a new database, paste and run.

```
-- Run once in SQL Server (2016+). IS NOT DISTINCT FROM in Q14 needs 2022.
CREATE TABLE Departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(30),
    city       VARCHAR(30)
);
INSERT INTO Departments VALUES
 (10,'Engineering','Jaipur'), (20,'Sales','Delhi'),
 (30,'HR','Pune'),            (40,'Finance','Mumbai');

CREATE TABLE Employees (
    emp_id     INT PRIMARY KEY,
    emp_name   VARCHAR(30),
    dept_id    INT NULL,          -- Kabir has no department
    manager_id INT NULL,          -- Aarav has no manager
    salary     INT
);
INSERT INTO Employees VALUES
 (101,'Aarav', 10,  NULL,150000), (102,'Priya', 10, 101, 95000),
 (103,'Rohan', 20,  101,  80000), (104,'Sneha', 20, 103, 85000),
 (105,'Kabir', NULL,101,  55000), (106,'Meera', 30, 101, 70000),
 (107,'Ishaan',10,  102,  95000);

CREATE TABLE Projects (project_id INT PRIMARY KEY, project_name VARCHAR(30));
INSERT INTO Projects VALUES (1,'Data Lake'),(2,'CRM Revamp'),(3,'Payroll Bot');

CREATE TABLE EmployeeProjects (emp_id INT, project_id INT);
INSERT INTO EmployeeProjects VALUES
 (101,1),(101,2),(101,3),(102,1),(103,2),
 (104,2),(104,3),(107,1),(107,2),(107,3);

CREATE TABLE SalaryGrades (grade CHAR(1), min_sal INT, max_sal INT);
INSERT INTO SalaryGrades VALUES
 ('A',100000,200000),('B',75000,99999),('C',50000,74999);

CREATE TABLE OrderItems (order_id INT, item VARCHAR(20), amount INT);
INSERT INTO OrderItems VALUES
 (5001,'Laptop bag',300),(5001,'Mouse',200),(5002,'Monitor',400);

CREATE TABLE Payments (payment_id INT, order_id INT, paid INT);
INSERT INTO Payments VALUES (1,5001,100),(2,5001,150),(3,5001,250),(4,5002,400);

CREATE TABLE T1 (val INT NULL);
INSERT INTO T1 VALUES (1),(1),(2),(NULL);
CREATE TABLE T2 (val INT NULL);
INSERT INTO T2 VALUES (1),(1),(1),(3),(NULL);
```

How SQL Server processes a join

You write SELECT first, but SQL Server logically evaluates it almost last. This order explains most join "mysteries" in this guide.

1	**FROM**	Read the first table
2	**ON**	Pair rows and keep pairs where the condition is TRUE
3	**JOIN (outer)**	LEFT/RIGHT/FULL add back unmatched rows with NULLs
4	**WHERE**	Filter rows (runs after outer rows were added)
5	**GROUP BY**	Form groups
6	**HAVING**	Filter groups
7	**SELECT**	Compute columns, aliases are born here
8	**DISTINCT**	Remove duplicate rows
9	**ORDER BY**	Sort (the only step that can use SELECT aliases)
10	**TOP**	Keep the first N rows

The three phases of every join

1. **Cartesian product.** Conceptually, every left row is paired with every right row.
2. **ON filter.** Only pairs where the ON condition is TRUE survive. FALSE and UNKNOWN (anything compared with NULL) are both thrown away.
3. **Add outer rows.** For LEFT, RIGHT and FULL joins, preserved rows that found no partner are added back with NULLs. INNER and CROSS joins skip this phase.

This is a logical model, not what the engine physically does. The optimizer never builds the full product when it can avoid it (see Q24). But the result is always as if these steps happened in this order.

Join cheat sheet

Join	Keeps	Row count
INNER	Matching pairs only	matches
LEFT	All left rows	matches + unmatched left
RIGHT	All right rows	matches + unmatched right
FULL	Everything	matches + unmatched left + unmatched right
CROSS	Every combination	left rows × right rows
SELF	Table joined to itself	depends on the join type used
CROSS APPLY	Left rows where inner query returns rows	per-row result of inner query
OUTER APPLY	All left rows	as CROSS APPLY + left rows with no result

■□□

Easy

The core join types, read correctly. These questions test whether you know exactly which rows survive, including NULLs.

Q01	INNER JOIN — employees with their department
Q02	LEFT JOIN — keep every employee
Q03	Find departments with no employees
Q04	RIGHT JOIN — and why most people rewrite it
Q05	FULL OUTER JOIN — nothing gets lost
Q06	CROSS JOIN — every combination
Q07	SELF JOIN — employee and manager name
Q08	Count employees per department, including zero

Q01 ■□□ EASY

INNER JOIN — employees with their department

Show every employee's name with their department name. Employees without a department should not appear.

QUERY

```
SELECT e.emp_name, d.dept_name
FROM Employees AS e
INNER JOIN Departments AS d
        ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

OUTPUT · 6 ROWS

emp_name	dept_name
Aarav	Engineering
Priya	Engineering
Rohan	Sales
Sneha	Sales
Meera	HR
Ishaan	Engineering

HOW IT EXECUTES

1. FROM: SQL Server conceptually pairs every Employees row with every Departments row (7 × 4 = 28 pairs).
2. ON: keeps only pairs where e.dept_id = d.dept_id. Kabir's dept_id is NULL, and NULL = 10 is UNKNOWN, not TRUE, so Kabir is dropped. 6 rows survive.
3. SELECT: picks emp_name and dept_name. ORDER BY emp_id runs last.
4. Finance (40) has no employees, so it never appears either. INNER JOIN keeps matches only, from both sides.

> **Trap.** Kabir disappears silently. In real data, NULL foreign keys make INNER JOIN 'lose' rows and nobody notices until totals don't match.

> **Interviewer follow-up.** Does writing JOIN instead of INNER JOIN change anything? No. INNER is the default and optional.

Q02 ■□□ EASY

LEFT JOIN — keep every employee

List all employees with their department. If an employee has no department, show 'Unassigned'.

QUERY

```
SELECT e.emp_name,
       ISNULL(d.dept_name, 'Unassigned') AS dept_name
FROM Employees AS e
LEFT JOIN Departments AS d
       ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

OUTPUT · 7 ROWS

emp_name	dept_name
Aarav	Engineering
Priya	Engineering
Rohan	Sales
Sneha	Sales
Kabir	Unassigned
Meera	HR
Ishaan	Engineering

HOW IT EXECUTES

1. Phase 1, Cartesian: 7 × 4 = 28 pairs.
2. Phase 2, ON filter: 6 matching pairs survive (same as INNER JOIN).
3. Phase 3, add outer rows: every LEFT table row that found no match is added back with NULLs for the right table. Kabir comes back with dept_name = NULL. Total 7.
4. SELECT: ISNULL turns Kabir's NULL into 'Unassigned'.

> **Trap.** LEFT JOIN = INNER JOIN + unmatched rows from the left table. That 'phase 3' is the whole difference, and it happens before WHERE runs (see Q10).

> **Interviewer follow-up.** ISNULL vs COALESCE? ISNULL takes 2 arguments and returns the first argument's data type. COALESCE takes many and follows data-type precedence.

Q03 ■□□ EASY

Find departments with no employees

Which departments currently have no employees?

QUERY

```
SELECT d.dept_name
FROM Departments AS d
LEFT JOIN Employees AS e
       ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

OUTPUT · 1 ROW

dept_name
Finance

HOW IT EXECUTES

1. LEFT JOIN keeps all 4 departments. Engineering, Sales and HR match employees (6 rows); Finance has no match, so it comes back once with all Employees columns NULL. Total 7 rows.
2. WHERE e.emp_id IS NULL keeps only the 'no match' row: Finance.
3. Test a column that can never be NULL in a real match (the primary key emp_id), not a nullable column like salary.

> **Trap.** This pattern is called an anti-join. Checking a nullable column (e.g. e.manager_id IS NULL) would wrongly return a department whose matched employee simply has a NULL manager.

> **Interviewer follow-up.** Write the same thing with NOT EXISTS. Then with NOT IN, and explain why NOT IN breaks here (answer in Q22).

Q04 ■□□ EASY

RIGHT JOIN — and why most people rewrite it

Show all departments with their employees, including departments that have nobody. Write it with RIGHT JOIN, then rewrite it as a LEFT JOIN.

QUERY

```sql
-- RIGHT JOIN version
SELECT d.dept_name, e.emp_name
FROM Employees AS e
RIGHT JOIN Departments AS d
        ON e.dept_id = d.dept_id
ORDER BY d.dept_id, e.emp_id;

-- Same result, LEFT JOIN version (preferred)
SELECT d.dept_name, e.emp_name
FROM Departments AS d
LEFT JOIN Employees AS e
       ON e.dept_id = d.dept_id
ORDER BY d.dept_id, e.emp_id;
```

OUTPUT · 7 ROWS

dept_name	emp_name
Engineering	Aarav
Engineering	Priya
Engineering	Ishaan
Sales	Rohan
Sales	Sneha
HR	Meera
Finance	*NULL*

HOW IT EXECUTES

1. RIGHT JOIN preserves the table written on the RIGHT (Departments). Every department is kept; Finance comes back with emp_name NULL.
2. Kabir (no department) is not in the result: he belongs to the non-preserved side.
3. A RIGHT JOIN B is exactly B LEFT JOIN A. Swap the table order and the keyword.

> **Trap.** SQL Server's optimizer often flips them internally anyway. In an execution plan you may see 'Right Outer Join' even though you wrote LEFT, because it picked the other table as the outer input.

> **Interviewer follow-up.** Why do teams avoid RIGHT JOIN? Readability: with a chain of joins, mixing LEFT and RIGHT makes it hard to see which table is preserved.

Q05 ■□□ EASY

FULL OUTER JOIN — nothing gets lost

Show every employee and every department in one result, matched where possible.

QUERY

```
SELECT e.emp_name, d.dept_name
FROM Employees AS e
FULL OUTER JOIN Departments AS d
        ON e.dept_id = d.dept_id
ORDER BY d.dept_id, e.emp_id;
```

OUTPUT · 8 ROWS

emp_name	dept_name
Kabir	*NULL*
Aarav	Engineering
Priya	Engineering
Ishaan	Engineering
Rohan	Sales
Sneha	Sales
Meera	HR
NULL	Finance

HOW IT EXECUTES

1. ON finds the 6 matching pairs.
2. Unmatched rows from the LEFT side are added (Kabir, with dept_name NULL).
3. Unmatched rows from the RIGHT side are added (Finance, with emp_name NULL).
4. 6 + 1 + 1 = 8 rows. In SQL Server, NULLs sort first in ascending order, so Kabir's row (dept_id NULL) is on top.

> **Trap.** Rule of thumb: FULL rows = matched pairs + unmatched left rows + unmatched right rows. Interviewers love asking for this number without running it.

> **Interviewer follow-up.** How would you show only the mismatches? Add WHERE e.emp_id IS NULL OR d.dept_id IS NULL. Useful for data reconciliation.

Q06 ■□□ EASY

CROSS JOIN — every combination

How many rows does Employees CROSS JOIN Departments return? Write the query.

QUERY

```
SELECT COUNT(*) AS total_rows
FROM Employees AS e
CROSS JOIN Departments AS d;
```

OUTPUT · 1 ROW

total_rows
28

HOW IT EXECUTES

1. CROSS JOIN has no ON clause. It returns the Cartesian product: every left row paired with every right row.
2. 7 employees × 4 departments = 28 rows. NULLs do not matter here; Kabir still pairs with all 4 departments.
3. Every other join starts from this product conceptually, then filters it with ON.

> **Trap.** FROM Employees, Departments (comma, no WHERE) is also a CROSS JOIN. Forgetting the WHERE in old comma-style joins is a classic way to accidentally return millions of rows.

> **Interviewer follow-up.** Real use? Generating combinations: every store × every date for a sales calendar, every size × every colour for a product catalogue.

Q07 ■□□ EASY

SELF JOIN — employee and manager name

Show each employee with their manager's name. The top boss should show 'No Manager'.

QUERY

```
SELECT e.emp_name AS employee,
       ISNULL(m.emp_name, 'No Manager') AS manager
FROM Employees AS e
LEFT JOIN Employees AS m
       ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

OUTPUT · 7 ROWS

employee	manager
Aarav	No Manager
Priya	Aarav
Rohan	Aarav
Sneha	Rohan
Kabir	Aarav
Meera	Aarav
Ishaan	Priya

HOW IT EXECUTES

1. The same table is opened twice under two aliases: e plays 'employee', m plays 'manager'.
2. ON e.manager_id = m.emp_id looks up each employee's manager row.
3. Aarav's manager_id is NULL, so he finds no match. LEFT JOIN keeps him; ISNULL prints 'No Manager'.

> **Trap.** With INNER JOIN, the CEO vanishes. In a self join, think about the top and bottom of the hierarchy every time.

> **Interviewer follow-up.** Aliases are mandatory in a self join. Without them SQL Server cannot tell which copy of Employees a column belongs to.

Q08 ■□□ EASY

Count employees per department, including zero

Show the number of employees in each department. Departments with no employees must show 0.

QUERY

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS emp_count
FROM Departments AS d
LEFT JOIN Employees AS e
       ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY emp_count DESC, d.dept_name;
```

OUTPUT · 4 ROWS

dept_name	emp_count
Engineering	3
Sales	2
HR	1
Finance	0

HOW IT EXECUTES

1. LEFT JOIN returns 7 rows: 6 matched pairs plus Finance with NULL employee columns.
2. GROUP BY dept_name makes 4 groups.
3. COUNT(e.emp_id) counts only non-NULL values. Finance's single row has emp_id NULL, so its count is 0.

> **Trap.** COUNT(*) counts rows, not employees. It would give Finance = 1, because the outer row from the LEFT JOIN is still a row. This is one of the most common interview mistakes.

> **Interviewer follow-up.** Why GROUP BY dept_name and not dept_id? Either works here; in production group by the key (dept_id) as well, since two departments could share a name.

■■□

Intermediate

Row-count puzzles, ON vs WHERE, self joins, NULL-safe and non-equi joins.
Where most candidates lose marks.

Q09	The duplicate + NULL row-count puzzle
Q10	ON vs WHERE in a LEFT JOIN
Q11	Employees who earn more than their manager
Q12	Highest-paid employee in each department
Q13	Chaining joins: one INNER JOIN breaks your LEFT JOINs
Q14	Joining when the key can be NULL
Q15	Non-equi join — salary grades
Q16	Pairs of employees in the same department

Q09 ■■□ INTERMEDIATE

The duplicate + NULL row-count puzzle

T1(val) = 1, 1, 2, NULL. T2(val) = 1, 1, 1, 3, NULL. Without running anything, how many rows does each join return on T1.val = T2.val?

QUERY

```
SELECT 'INNER' AS join_type, COUNT(*) AS row_count
FROM T1 INNER JOIN T2 ON T1.val = T2.val
UNION ALL
SELECT 'LEFT',  COUNT(*) FROM T1 LEFT  JOIN T2 ON T1.val = T2.val
UNION ALL
SELECT 'RIGHT', COUNT(*) FROM T1 RIGHT JOIN T2 ON T1.val = T2.val
UNION ALL
SELECT 'FULL',  COUNT(*) FROM T1 FULL  JOIN T2 ON T1.val = T2.val
UNION ALL
SELECT 'CROSS', COUNT(*) FROM T1 CROSS JOIN T2;
```

OUTPUT · 5 ROWS

join_type	row_count
INNER	6
LEFT	8
RIGHT	8
FULL	10
CROSS	20

HOW IT EXECUTES

1. Value 1: two 1s in T1 × three 1s in T2 = 6 pairs. That is the whole INNER JOIN. Duplicates multiply.
2. NULL never equals NULL (the result is UNKNOWN), so the two NULLs never match. 2 and 3 have no partner.
3. LEFT = 6 + unmatched T1 rows (2, NULL) = 8. RIGHT = 6 + unmatched T2 rows (3, NULL) = 8.
4. FULL = 6 + 2 + 2 = 10. CROSS = 4 × 5 = 20.

> **Trap.** Most candidates say INNER = 2 or 3. Remember: matches multiply (2 × 3), they do not deduplicate.

> **Interviewer follow-up.** What if both tables had a single NULL and you need them to match? See Q14.

Q10 ■■□ INTERMEDIATE

ON vs WHERE in a LEFT JOIN

List every department with its employees earning more than 80,000. Departments with no such employee must still appear.

QUERY

```sql
-- Correct: filter inside ON
SELECT d.dept_name, e.emp_name, e.salary
FROM Departments AS d
LEFT JOIN Employees AS e
       ON e.dept_id = d.dept_id
      AND e.salary > 80000
ORDER BY d.dept_id, e.emp_id;

-- Wrong: filter in WHERE (HR and Finance vanish)
-- ... LEFT JOIN Employees e ON e.dept_id = d.dept_id
-- WHERE e.salary > 80000
```

OUTPUT · 6 ROWS

dept_name	emp_name	salary
Engineering	Aarav	150,000
Engineering	Priya	95,000
Engineering	Ishaan	95,000
Sales	Sneha	85,000
HR	*NULL*	*NULL*
Finance	*NULL*	*NULL*

HOW IT EXECUTES

1. Filter in ON: the condition decides which employees MATCH. HR's only employee (Meera, 70,000) fails it, so HR has no match and is added back in phase 3 with NULLs. Same for Finance. 6 rows.
2. Filter in WHERE: the LEFT JOIN finishes first (phase 3 already added outer rows), THEN WHERE runs. For HR and Finance, e.salary is NULL, NULL > 80000 is UNKNOWN, the row is removed. Only 4 rows.
3. A WHERE condition on the right table of a LEFT JOIN silently turns it into an INNER JOIN.

> **Trap.** For INNER JOIN, ON vs WHERE gives the same result. For OUTER joins it changes the answer. Say this sentence in the interview.

> **Interviewer follow-up.** When IS a WHERE filter on the right table correct? When you test for IS NULL (anti-join, Q3), because you are filtering on the outer rows on purpose.

Q11 ■■□ INTERMEDIATE

Employees who earn more than their manager

Find employees whose salary is strictly greater than their manager's salary.

QUERY

```
SELECT e.emp_name AS employee, e.salary,
       m.emp_name AS manager,  m.salary AS manager_salary
FROM Employees AS e
INNER JOIN Employees AS m
        ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

OUTPUT · 1 ROW

employee	salary	manager	manager_salary
Sneha	85,000	Rohan	80,000

HOW IT EXECUTES

1. Self join pairs each employee with their manager: 6 pairs (Aarav has no manager, INNER JOIN drops him; correct here).
2. WHERE compares the two salaries on the same row: Sneha 85,000 > Rohan 80,000. True.
3. Ishaan 95,000 vs Priya 95,000 is equal, not greater, so he is excluded.

> **Trap.** Ties. If the question says 'at least as much', use >=. Always ask the interviewer how ties should be handled.

> **Interviewer follow-up.** Count, for each manager, how many reports earn more than them: GROUP BY m.emp_name over the same join.

Q12 ■■□ INTERMEDIATE

Highest-paid employee in each department

Show the top earner of every department (keep ties), using a JOIN, not a window function.

QUERY

```
SELECT d.dept_name, e.emp_name, e.salary
FROM Employees AS e
INNER JOIN (SELECT dept_id, MAX(salary) AS max_sal
            FROM Employees
            GROUP BY dept_id) AS mx
        ON  e.dept_id = mx.dept_id
        AND e.salary  = mx.max_sal
INNER JOIN Departments AS d
        ON d.dept_id = e.dept_id
ORDER BY d.dept_id;
```

OUTPUT · 3 ROWS

dept_name	emp_name	salary
Engineering	Aarav	150,000
Sales	Sneha	85,000
HR	Meera	70,000

HOW IT EXECUTES

1. Derived table mx runs first: one row per dept_id with its MAX salary. It includes a NULL group (Kabir, 55,000).
2. Join back to Employees on BOTH dept_id and salary. Anyone matching the max is kept, so ties are kept automatically.
3. The NULL group never matches (NULL = NULL is UNKNOWN), so Kabir drops out. Department-less employees are not 'top of a department'.
4. Join Departments for the name. Result: 3 rows. Finance has no employees, so no row.

> **Trap.** Joining only on salary (without dept_id) is wrong: a 95,000 earner in one department would match the 95,000 max of another.

> **Interviewer follow-up.** Now do it with DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) = 1, and explain why ROW_NUMBER would drop ties.

Q13 ■■□ INTERMEDIATE

Chaining joins: one INNER JOIN breaks your LEFT JOINs

For every employee, show department (or 'Unassigned') and number of projects. Employees with no projects must show 0.

QUERY

```
SELECT e.emp_name,
       ISNULL(d.dept_name, 'Unassigned') AS dept_name,
       COUNT(p.project_id)               AS projects
FROM Employees AS e
LEFT JOIN Departments      AS d  ON d.dept_id    = e.dept_id
LEFT JOIN EmployeeProjects AS ep ON ep.emp_id    = e.emp_id
LEFT JOIN Projects         AS p  ON p.project_id = ep.project_id
GROUP BY e.emp_id, e.emp_name, d.dept_name
ORDER BY e.emp_id;
```

OUTPUT · 7 ROWS

emp_name	dept_name	projects
Aarav	Engineering	3
Priya	Engineering	1
Rohan	Sales	1
Sneha	Sales	2
Kabir	Unassigned	0
Meera	HR	0
Ishaan	Engineering	3

HOW IT EXECUTES

1. Joins are processed left to right. The result of each join becomes the left input of the next.
2. Join 1 keeps all 7 employees. Join 2 multiplies rows by project assignments (Aarav becomes 3 rows) and keeps Kabir and Meera with NULLs. Join 3 adds project details.
3. GROUP BY collapses back to one row per employee; COUNT(p.project_id) ignores NULLs, giving 0 for Kabir and Meera.

> **Trap.** Change the last join to INNER JOIN Projects and Kabir and Meera disappear: their p.project_id is NULL, so the INNER join's ON fails and removes rows the earlier LEFT JOINs had kept. Once you start a LEFT chain, keep it LEFT.

> **Interviewer follow-up.** Why GROUP BY e.emp_id too? Two employees could share a name; grouping by the key keeps them separate.

Joining when the key can be NULL

In the T1 / T2 tables from Q9, treat NULL as a real value so NULL matches NULL. How many rows does the INNER JOIN return now?

QUERY

```sql
-- SQL Server 2022+
SELECT COUNT(*) AS row_count
FROM T1
INNER JOIN T2
        ON T1.val IS NOT DISTINCT FROM T2.val;

-- Any version
SELECT COUNT(*) AS row_count
FROM T1
INNER JOIN T2
        ON T1.val = T2.val
        OR (T1.val IS NULL AND T2.val IS NULL);
```

OUTPUT · 1 ROW

row_count
7

HOW IT EXECUTES

1. Normal '=' : 6 rows (only the 1s match, Q9).
2. IS NOT DISTINCT FROM treats two NULLs as equal: the single NULL in T1 now matches the single NULL in T2. 6 + 1 = 7.
3. The OR version gives the same answer on older versions.

> **Trap.** Avoid ON ISNULL(T1.val, -1) = ISNULL(T2.val, -1). It works only if -1 never appears in real data, and wrapping columns in functions blocks index seeks (non-SARGable).

> **Interviewer follow-up.** What does SET ANSI_NULLS OFF do? It makes '= NULL' comparisons true in some cases. It is deprecated; never rely on it.

Q15 ■■□ INTERMEDIATE

Non-equi join — salary grades

Assign each employee a grade from SalaryGrades, where salary falls between min_sal and max_sal.

QUERY

```
SELECT e.emp_name, e.salary, g.grade
FROM Employees AS e
INNER JOIN SalaryGrades AS g
        ON e.salary BETWEEN g.min_sal AND g.max_sal
ORDER BY e.salary DESC, e.emp_id;
```

OUTPUT · 7 ROWS

emp_name	salary	grade
Aarav	150,000	A
Priya	95,000	B
Ishaan	95,000	B
Sneha	85,000	B
Rohan	80,000	B
Meera	70,000	C
Kabir	55,000	C

HOW IT EXECUTES

1. A join condition does not have to be '='. Here each employee row is tested against each grade range.
2. BETWEEN is inclusive on both ends: 75,000 and 99,999 both count as grade B.
3. Each salary falls in exactly one range, so 7 rows come back, one per employee.

> **Trap.** If ranges overlap, an employee matches two grades and is duplicated. If there is a gap (e.g. 99,999.50 with DECIMAL salaries), the employee vanishes. Always check ranges are contiguous and non-overlapping.

> **Interviewer follow-up.** Which physical join can SQL Server use for a pure range condition? Only Nested Loops; Hash and Merge need at least one equality predicate (see Q24).

Q16 ■■□ INTERMEDIATE

Pairs of employees in the same department

List every pair of colleagues who work in the same department. Each pair should appear once.

QUERY

```
SELECT d.dept_name, a.emp_name AS emp_1, b.emp_name AS emp_2
FROM Employees AS a
INNER JOIN Employees AS b
        ON  a.dept_id = b.dept_id
        AND a.emp_id  < b.emp_id
INNER JOIN Departments AS d
        ON d.dept_id = a.dept_id
ORDER BY d.dept_id, a.emp_id, b.emp_id;
```

OUTPUT · 4 ROWS

dept_name	emp_1	emp_2
Engineering	Aarav	Priya
Engineering	Aarav	Ishaan
Engineering	Priya	Ishaan
Sales	Rohan	Sneha

HOW IT EXECUTES

1. Self join on dept_id pairs every employee with everyone in their department, including themselves.
2. a.emp_id < b.emp_id removes self-pairs (Priya-Priya) and mirror pairs (Priya-Aarav when Aarav-Priya exists).
3. Engineering has 3 people, so 3 pairs; Sales has 2, so 1 pair; HR has 1, so none. Total 4.

> **Trap.** With a.emp_id <> b.emp_id you get 8 rows (every pair twice). With no second condition you get 14 (every pair twice plus 6 self-pairs, one per employee in a department). Interviewers check whether you know the difference.

> **Interviewer follow-up.** Formula: a department of n people gives n(n−1)/2 unique pairs.

Hard

Problems that look simple but break in production: fan-out, the NOT IN trap, APPLY, recursion, relational division and physical join operators.

Q17	Employees above their department average
Q18	Second-highest salary per department (no window functions)
Q19	CROSS APPLY vs OUTER APPLY — top earner per department
Q20	Recursive self join — full reporting chain
Q21	The fan-out trap — inflated SUMs
Q22	NOT IN vs NOT EXISTS vs LEFT JOIN — the NULL trap
Q23	Relational division — employees on ALL projects
Q24	Logical vs physical joins — what SQL Server really runs

Q17 ■■■ HARD

Employees above their department average

Find employees who earn more than the average salary of their own department. Show the department average too.

QUERY

```
SELECT e.emp_name, d.dept_name, e.salary, a.dept_avg
FROM Employees AS e
INNER JOIN (SELECT dept_id, AVG(salary) AS dept_avg
            FROM Employees
            GROUP BY dept_id) AS a
        ON a.dept_id = e.dept_id
INNER JOIN Departments AS d
        ON d.dept_id = e.dept_id
WHERE e.salary > a.dept_avg
ORDER BY d.dept_id;
```

OUTPUT · 2 ROWS

emp_name	dept_name	salary	dept_avg
Aarav	Engineering	150,000	113,333
Sneha	Sales	85,000	82,500

HOW IT EXECUTES

1. The derived table computes one average per department: Engineering, Sales, HR (and a NULL group for Kabir, which never joins).
2. Joining it back puts each employee's department average on their own row, so WHERE can compare two columns.
3. Engineering avg 113,333: only Aarav is above. Sales avg 82,500: Sneha. HR avg 70,000: Meera is equal, not above.

> **Trap.** AVG on an INT column returns an INT in SQL Server: 340,000 / 3 shows 113,333, not 113,333.33. Use AVG(salary * 1.0) or AVG(CAST(salary AS DECIMAL(12,2))) when decimals matter.

> **Interviewer follow-up.** Rewrite with AVG(salary) OVER (PARTITION BY dept_id). Which one reads the table once? The window version, usually.

Q18 ■■■ HARD

Second-highest salary per department (no window functions)

Using only joins and GROUP BY, find the employees with the second-highest distinct salary in their department.

QUERY

```
SELECT d.dept_name, e1.emp_name, e1.salary
FROM Employees AS e1
LEFT JOIN Employees AS e2
       ON  e2.dept_id = e1.dept_id
       AND e2.salary  > e1.salary
INNER JOIN Departments AS d
        ON d.dept_id = e1.dept_id
GROUP BY d.dept_id, d.dept_name, e1.emp_id, e1.emp_name, e1.salary
HAVING COUNT(DISTINCT e2.salary) = 1
ORDER BY d.dept_id, e1.emp_id;
```

OUTPUT · 3 ROWS

dept_name	emp_name	salary
Engineering	Priya	95,000
Engineering	Ishaan	95,000
Sales	Rohan	80,000

HOW IT EXECUTES

1. For each employee e1, the LEFT self join brings every colleague e2 in the same department who earns MORE.
2. COUNT(DISTINCT e2.salary) = how many distinct salaries are above you. 0 means you are the top; 1 means you are second.
3. Engineering: Priya and Ishaan (95,000) each have exactly one higher distinct salary (150,000). Both returned: ties handled.
4. Sales: Rohan (80,000) is below 85,000. HR has one person, so no second place.

> **Trap.** DISTINCT is the key. Without it, if two people earned 150,000, the 95,000 earners would count 2 higher rows and be missed.

> **Interviewer follow-up.** Generalise to the Nth highest: HAVING COUNT(DISTINCT e2.salary) = N − 1. Equivalent window version: DENSE_RANK() = N.

Q19 ■■■ HARD

CROSS APPLY vs OUTER APPLY — top earner per department

For every department, return its single highest-paid employee. Departments with nobody must still appear.

QUERY

```
SELECT d.dept_name, t.emp_name, t.salary
FROM Departments AS d
OUTER APPLY (SELECT TOP (1) e.emp_name, e.salary
             FROM Employees AS e
             WHERE e.dept_id = d.dept_id
             ORDER BY e.salary DESC, e.emp_id) AS t
ORDER BY d.dept_id;
```

OUTPUT · 4 ROWS

dept_name	emp_name	salary
Engineering	Aarav	150,000
Sales	Sneha	85,000
HR	Meera	70,000
Finance	*NULL*	*NULL*

HOW IT EXECUTES

1. APPLY runs the inner query once per outer row, and the inner query can use the outer row's columns (d.dept_id). A normal JOIN cannot do that with a derived table.
2. For each department, TOP (1) … ORDER BY salary DESC picks the top earner. emp_id is the tie-breaker so the result is deterministic.
3. OUTER APPLY works like LEFT JOIN: Finance returns nothing from the inner query but is kept with NULLs. CROSS APPLY works like INNER JOIN and would drop Finance (3 rows).

> **Trap.** TOP without a unique ORDER BY is non-deterministic: with ties, SQL Server may return a different row each run. Always add a tie-breaker.

> **Interviewer follow-up.** Use APPLY for 'top N per group' with an index on (dept_id, salary DESC): SQL Server can seek and read just N rows per group.

Q20 ■■■ HARD

Recursive self join — full reporting chain

Show every employee with their level in the hierarchy (CEO = 1) and their full path from the top.

QUERY

```
WITH Chain AS (
    SELECT emp_id, emp_name, 1 AS lvl,
           CAST(emp_name AS VARCHAR(200)) AS path
    FROM Employees
    WHERE manager_id IS NULL                -- anchor
    UNION ALL
    SELECT e.emp_id, e.emp_name, c.lvl + 1,
           CAST(c.path + ' > ' + e.emp_name AS VARCHAR(200))
    FROM Employees AS e
    INNER JOIN Chain AS c                   -- recursive join
            ON e.manager_id = c.emp_id
)
SELECT emp_name, lvl, path
FROM Chain
ORDER BY path
OPTION (MAXRECURSION 100);
```

OUTPUT · 7 ROWS

emp_name	lvl	path
Aarav	1	Aarav
Kabir	2	Aarav > Kabir
Meera	2	Aarav > Meera
Priya	2	Aarav > Priya
Ishaan	3	Aarav > Priya > Ishaan
Rohan	2	Aarav > Rohan
Sneha	3	Aarav > Rohan > Sneha

HOW IT EXECUTES

1. Anchor runs once: the employee with no manager (Aarav), level 1.
2. Recursive part joins Employees to the rows produced in the previous round: round 2 finds Priya, Rohan, Kabir, Meera (level 2); round 3 finds Ishaan and Sneha (level 3).
3. Round 4 finds nobody, the recursion stops, and all rounds are combined with UNION ALL.

> **Trap.** Without the CAST, SQL Server raises 'Types don't match between the anchor and the recursive part': emp_name is VARCHAR(30) but the concatenated path is longer. Both parts must return identical types.

> **Interviewer follow-up.** What stops an infinite loop if data has a cycle (A manages B, B manages A)? MAXRECURSION (default 100) raises an error. Detect cycles by checking the path before joining.

Q21 ■■■ HARD

The fan-out trap — inflated SUMs

For each order, show total item amount and total paid. Order 5001 has items worth 500 and payments of 500.

QUERY

```
-- WRONG: joins two one-to-many tables directly
SELECT i.order_id,
       SUM(i.amount) AS billed,
       SUM(p.paid)   AS paid
FROM OrderItems AS i
INNER JOIN Payments AS p ON p.order_id = i.order_id
GROUP BY i.order_id;
-- 5001 → billed 1500, paid 1000  (both wrong)

-- RIGHT: aggregate each side first, then join
WITH b AS (SELECT order_id, SUM(amount) AS billed
           FROM OrderItems GROUP BY order_id),
     p AS (SELECT order_id, SUM(paid) AS paid
           FROM Payments   GROUP BY order_id)
SELECT b.order_id, b.billed, p.paid
FROM b INNER JOIN p ON p.order_id = b.order_id
ORDER BY b.order_id;
```

OUTPUT · 2 ROWS

order_id	billed	paid
5001	500	500
5002	400	400

HOW IT EXECUTES

1. Order 5001 has 2 items and 3 payments. Joining them on order_id pairs every item with every payment: 2 × 3 = 6 rows.
2. Each item amount now appears 3 times (500 × 3 = 1500); each payment appears 2 times (500 × 2 = 1000).
3. Fix: reduce each table to ONE row per order first (CTEs b and p), then join 1-to-1. Nothing multiplies.

> **Trap.** This bug passes testing whenever each order happens to have one item or one payment. It shows up in production reports. Check row counts before and after every join.

> **Interviewer follow-up.** If some orders have no payments, which join do you use in the fixed version? LEFT JOIN p, with ISNULL(p.paid, 0).

Q22 ■■■ HARD

NOT IN vs NOT EXISTS vs LEFT JOIN — the NULL trap

Find departments with no employees using NOT IN. Explain the result, then fix it.

QUERY

```sql
-- 1) NOT IN: returns ZERO rows!
SELECT dept_name FROM Departments
WHERE dept_id NOT IN (SELECT dept_id FROM Employees);

-- 2) NOT EXISTS: returns Finance
SELECT d.dept_name FROM Departments AS d
WHERE NOT EXISTS (SELECT 1 FROM Employees AS e
                  WHERE e.dept_id = d.dept_id);

-- 3) LEFT JOIN anti-join: returns Finance
SELECT d.dept_name FROM Departments AS d
LEFT JOIN Employees AS e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

OUTPUT · 1 ROW

dept_name
Finance

HOW IT EXECUTES

1. The subquery returns 10, 10, 20, 20, NULL, 30, 10. It contains a NULL (Kabir).
2. For Finance, 40 NOT IN (10, 20, NULL, 30) means 40<>10 AND 40<>20 AND 40<>NULL AND 40<>30. 40<>NULL is UNKNOWN, so the whole AND is UNKNOWN, never TRUE. Every row is rejected.
3. NOT EXISTS only asks 'is there a matching row?'. A NULL dept_id never matches 40, so Finance is correctly returned. The LEFT JOIN version behaves the same.

> **Trap.** One NULL in a NOT IN list wipes out the whole result, with no error. Prefer NOT EXISTS, or add WHERE dept_id IS NOT NULL inside the subquery.

> **Interviewer follow-up.** Which is fastest? In SQL Server, NOT EXISTS and LEFT JOIN ... IS NULL usually produce similar anti-semi-join plans; NOT EXISTS states the intent most clearly.

Q23 ■■■ HARD

Relational division — employees on ALL projects

Find employees who are assigned to every project in the Projects table.

QUERY

```
SELECT e.emp_name
FROM Employees AS e
INNER JOIN EmployeeProjects AS ep ON ep.emp_id = e.emp_id
GROUP BY e.emp_id, e.emp_name
HAVING COUNT(DISTINCT ep.project_id)
       = (SELECT COUNT(*) FROM Projects)
ORDER BY e.emp_id;
```

OUTPUT · 2 ROWS

emp_name
Aarav
Ishaan

HOW IT EXECUTES

1. The join gives one row per employee-project assignment (10 rows).
2. GROUP BY employee, then COUNT(DISTINCT project_id) = number of different projects each person is on.
3. HAVING compares that with the total project count (3). Aarav and Ishaan are on all 3.

> **Trap.** DISTINCT protects you from duplicate assignment rows. Also, if EmployeeProjects could contain a project_id that is not in Projects, join to Projects too, or the counts can lie.

> **Interviewer follow-up.** Classic alternative: 'there is no project this employee is NOT on', written with double NOT EXISTS.

Q24 ■■■ HARD

Logical vs physical joins — what SQL Server really runs

You wrote an INNER JOIN. The execution plan shows 'Hash Match'. Explain Nested Loops, Merge Join and Hash Match, and when the optimizer picks each.

QUERY

```
-- See the physical operator: Ctrl+M in SSMS (Include Actual Execution Plan)
SET STATISTICS IO, TIME ON;

SELECT e.emp_name, d.dept_name
FROM Employees AS e
INNER JOIN Departments AS d ON d.dept_id = e.dept_id;

-- Force an algorithm (for testing only, never as a default)
-- ... OPTION (LOOP JOIN);   OPTION (MERGE JOIN);   OPTION (HASH JOIN);
```

PHYSICAL JOIN OPERATORS

Operator	How it works	Best when	Needs
Nested Loops	For each outer row, search the inner input	Outer input small, inner side indexed on the join key	Nothing; the only one that handles pure non-equi joins
Merge Join	Walk two sorted inputs side by side	Both inputs already sorted on the key (e.g. clustered indexes)	At least one equality predicate; sorted inputs (or a Sort)
Hash Match	Build a hash table on the smaller input, probe with the larger	Large, unsorted inputs; no useful index	At least one equality predicate; memory grant (spills to tempdb)

HOW IT EXECUTES

1. INNER, LEFT, FULL are LOGICAL joins: they define the result. Nested Loops, Merge and Hash are PHYSICAL algorithms: they define how the result is produced. Any logical join can be run by different physical operators.
2. The optimizer chooses based on estimated row counts, available indexes and sort order. Bad statistics, wrong estimates, wrong operator: a classic cause of slow queries.
3. SQL Server 2017+ can also use an Adaptive Join (batch mode), which decides between Hash and Nested Loops at run time based on the actual row count.

> **Trap.** A join hint written inside the FROM clause (INNER HASH JOIN) also forces the join order of the whole query. Fix statistics and indexes first; use hints only to test a theory.

> **Interviewer follow-up.** Yellow warning on a Hash Match or Sort in the plan? Usually a spill to tempdb: the memory grant was too small because the row estimate was too low.

Rapid-fire round

Short questions that often come right after the main problem.

Does SQL Server support NATURAL JOIN or USING?

No. You must write ON explicitly.

Is LEFT OUTER JOIN different from LEFT JOIN?

No. OUTER is optional.

Can a LEFT JOIN return more rows than the left table?

Yes. If one left row matches several right rows, it repeats (Q9, Q13).

Does the order of tables matter in an INNER JOIN?

Not for the result. The optimizer reorders inner joins freely. For outer joins, order defines which side is preserved.

JOIN vs UNION?

JOIN adds columns (side by side). UNION adds rows (stacked) and needs matching column count and compatible types.

Joining an INT column to a VARCHAR column?

SQL Server converts implicitly by data-type precedence (VARCHAR to INT). It can fail on bad values and can block index seeks. Fix the types.

What is a semi-join?

Returns left rows that have at least one match, without duplicating them. EXISTS / IN are written semi-joins.

Can you join a table to a function?

Yes, with CROSS APPLY / OUTER APPLY to a table-valued function, passing outer columns as arguments.

Does a JOIN need a foreign key?

No. Foreign keys enforce integrity; joins only need comparable columns. A trusted FK can help the optimizer remove unneeded joins.

Why is SELECT * with joins risky?

Duplicate column names (dept_id twice), wider rows, and breaks when tables change. List the columns.

Revision checklist

Read this the night before the interview.

1. Matches multiply: 2 matching rows × 3 matching rows = 6.
2. NULL never equals NULL in ON. Use IS NOT DISTINCT FROM (2022+) or an explicit IS NULL check.
3. A WHERE filter on the right table of a LEFT JOIN turns it into an INNER JOIN. Put it in ON.
4. COUNT(column) ignores NULLs; COUNT(*) counts outer rows too.
5. One INNER JOIN after a chain of LEFT JOINs can remove the rows you meant to keep.
6. Never SUM across two one-to-many joins. Aggregate first, then join.
7. NOT IN with a NULL in the list returns nothing. Prefer NOT EXISTS.
8. In a self join, think about the top (no manager) and duplicate pairs (use <).
9. TOP needs a unique ORDER BY to be deterministic.
10. Logical join ≠ physical join. Know Nested Loops, Merge and Hash.

Data with Deepankar

Free to learn, free to share. If this helped, share it with someone preparing for a data interview.

Instagram **@datawithdeepankar**
YouTube **Deepankar Pathak**

NEXT IN THE SERIES

SQL Server Window Functions

Follow @datawithdeepankar so you don't miss it.